Injection-Aware Sentence Embeddings: A Method and Evaluation Audit for Prompt Injection Detection

Abdelghafour El Bikha

John Jay College

September 11, 2026

Abstract

Prompt-injection detectors can reject benign text that contains attack-like keywords. We describe Injection-Aware MPNet, a fine-tuned sentence encoder whose contrastive training samples injection–benign-trigger pairs twice as often as each other pair type, followed by an XGBoost classifier. An audit of the repository found that the historical encoder training pools overlap with benchmark inputs and with the encoder validation pool. The available results therefore do not establish performance on unseen inputs or a causal reduction in over-defense. We retain only explicitly labelled historical counts, distinguish the local synthetic trigger benchmark from official NotInject, and report a new, bounded single-seed re-evaluation on 2,315 held-out inputs. Doubling injection–benign-trigger pairs raises attack recall from 48.30% to 64.20% relative to uniform pairing, but also raises the false-positive rate from 25.50% to 36.78%. This experiment does not support a general reduction in over-defense. A separate supervised encoder pilot on 447 fresh BrowseSafe pages improves recall and false-positive rates for two training seeds, while a third seed degrades AUC and exceeds its validation false-positive budget on the holdout. A six-epoch follow-up with a disclosed mixed-precision repair improves recall for all three seeds on a separate 453-page holdout, reaching 66.36–69.12% recall at empirical FPR 2.54–4.66%. False positives increase for two seeds, and population FPR below 5% remains unestablished. Previously reported latency distributions, embedding-distance improvements, and comparative superiority remain withdrawn; fresh timings are reported with a limited measurement scope.

1 Introduction

Large Language Models (LLMs) deployed as conversational assistants and as components of agentic systems are exposed to a steady stream of untrusted text—direct user input, retrieved web pages, emails, and messages from other agents. Prompt injection attacks exploit this exposure by smuggling adversarial instructions into the input stream. While the attack surface and threat models in agentic deployments are still being mapped—e.g. Lee and Tiwari [2024] on cross-agent "prompt infection"—a robust input filter remains a foundational primitive for any defence stack.

Current input-side detectors face two recurring limitations:

- **Over-defense.** Detectors that lean on keyword matching or aggressive classifiers reject benign prompts that merely contain attack-adjacent terms ("system", "ignore", "bypass"), breaking legitimate use cases. Li and Liu [2024] showed that guardrail accuracy can fall to around 60% on the NotInject over-defense benchmark.
- **Lexical shortcuts.** Models trained on imbalanced corpora (where most malicious prompts contain trigger words and most benign prompts do not) may learn a trigger-word/malicious correlation rather than the intended distinction.

We investigate a focused detection approach:

- **Injection-Aware MPNet.** Cosine-similarity training of `all-mpnet-base-v2` with additional injection–benign-trigger pairs, intended to discourage reliance on shared keywords.
- **Embedding classification.** A 768-dimensional embedding feeds XGBoost; the historical operating threshold is 0.764.
- **Evaluation audit.** We trace the available numerical claims to source code and saved counts, and identify leakage, missing provenance, and measurements that require replacement.

The intended benefit of the pair weighting is a hypothesis. A controlled ablation and independent evaluation are needed to establish it. Coordination and response modules exist in the repository, but this paper makes no empirical claim about their protection of downstream agents.

2 Background & Related Work

Prompt Injection: Early work identified direct prompt injection [Perez and Ribeiro, 2022] and indirect injection via retrieved context [Greshake et al., 2023]. In multi-agent systems, "prompt infection" [Lee and Tiwari, 2024] describes how malicious prompts can replicate across agents.

Defense Mechanisms: Current defenses span multiple paradigms with distinct trade-offs:

2.1 Training-Time Defenses

StruQ [Chen et al., 2025a] separates prompts and data through a secure front-end and trains the LLM to follow instructions in the prompt channel. *SecAlign* [Chen et al., 2025b] uses preference optimization on secure and insecure outputs for injected inputs. Both modify the protected model through training; our approach instead trains an external detector. Their downstream attack-success metrics are not directly comparable to text-classification accuracy.

2.2 LLM-Based Defenses

PromptArmor [Shi et al., 2025] prompts an off-the-shelf LLM to detect and remove injected content before an agent processes it. Its paper reports false-positive and false-negative rates below 1% on AgentDojo with several guardrail models. We have not reproduced that experiment and make no comparative latency or superiority claim.

2.3 Over-Defense Mitigation

InjecGuard [Li and Liu, 2024] introduces NotInject, a 339-sample benign benchmark containing trigger words, and the Mitigating Over-defense for Free (MOF) training strategy. Our local synthetic trigger set is distinct from that benchmark. Both approaches target trigger-word bias, but matched data, model capacity, and evaluation conditions are needed to compare their training strategies.

3 Threat Model

Adversary capability. The adversary controls at least one untrusted text channel into the LLM application: a chat input, a fetched web page, an email body, a tool-call response, or a message from a peer agent in a multi-agent setup. The adversary's text is concatenated with system prompts and trusted context before reaching the LLM.

Adversary goals. (i) *Policy bypass*: cause the LLM to violate its system-level instructions. (ii) *Data exfiltration*: extract sensitive context (system prompts, conversation history,

tool outputs) into the response. (iii) *Lateral movement (multi-agent setting)*: have one agent issue commands that another agent then executes.

Defender capability. The defender places our detector inline between the untrusted text channel and the downstream LLM. The detector sees raw text, returns a binary decision plus a confidence score, and may strip or quarantine flagged inputs.

Out of scope.

- *Adversarial attacks on the LLM's parameters* (weight extraction, model inversion, supply-chain attacks on the base model).
- *Adaptive attacks on the detector itself* (gradient-based attacks such as GCG/AutoDAN). No model-matched adaptive evaluation is established here; see Section 9.
- *Multi-turn attacks that build context across many messages.* Our detector evaluates each message independently.

Scope. The primary focus of this paper is **single-turn, text-based prompt injection**. We additionally audit historical results for text extracted from HTML-embedded attacks (Section 7.4).

4 Detection System Design

The embedding classifier receives a text input, encodes it with MPNet, and applies XGBoost to the embedding. The returned injection score is thresholded to obtain a binary decision; calibration as a real-world probability has not been established. Historical records identify an embedding-only detector. The repository also contains regex matching and behavioural monitoring modules, but no matched experiment establishes their incremental accuracy or false-positive cost.

Multi-agent coordination and response. The repository contains guard, messaging, circuit-breaker, and quarantine modules. These are implementation components, not evidence of resistance to prompt propagation or downstream compromise. In particular, `OVONMessage.verify_integrity` accepts a claimed verification flag in message metadata rather than verifying an authenticated signature. Authentication and multi-agent effectiveness must be independently implemented and tested before a security claim is made. PeerGuard [Fan and Li, 2025] is related work on mutual reasoning against backdoors, not validation of these modules.

5 Methodology

5.1 Decision rule

For embedding $\mathbf{e}(x)$, let $s(x)$ be the XGBoost injection score. The historical classifier uses

$$\hat{y}(x) = \mathbf{1}[s(x) \geq 0.764]. \tag{1}$$

The classifier training entry point hard-codes this threshold. The saved metadata records it, but does not establish selection on a held-out validation set. We therefore do not describe it as optimal. A new operating threshold must be selected on validation data excluded from both training stages, then frozen before testing.

5.2 Injection-Aware MPNet contrastive fine-tuning

The historical encoder uses `all-mpnet-base-v2` and requests the following data. Loader calls are recorded in `training/finetune_mpnet_embeddings.py`.

- Injection text from SaTML (limit 1,500), deepset (up to 500 injections), and LLMail (limit 500).
- BrowseSafe (limit 2,000), whose extracted text is assigned to injection or safe pools according to its label.
- Safe deepset text (up to 1,000 safe samples).
- Official NotInject (limit 500) and local synthetic trigger text (limit 1,000), both treated as benign.

These are loader limits, not observed corpus sizes: the local synthetic loader generates only 247 prompts, and deepset's limit applies separately to each class. The original pipeline does not exclude benchmark text from these pools. Its validation pairs also draw from the last 200 entries of pools used for training. These overlaps invalidate an interpretation of its validation score as performance on unseen text.

Pair construction. From the loaded text, the script samples 14,000 `InputExample` pairs as in Table 1. The defining methodological choice is *doubling* the injection-vs. benign-with-trigger dissimilar pairs (4,000 vs. 2,000 of every other pair type), intended to emphasize prompts that share trigger words but differ in intent. This changes sampling frequency rather than assigning an explicit per-pair loss coefficient.

Table 1: Contrastive pair counts and target cosine similarity labels.

Pair type	Count	Target similarity
Injection–Injection	2,000	1.0
Safe–Safe	2,000	1.0
Benign-trigger–Benign-trigger	2,000	1.0
Injection–Safe	2,000	0.0
Injection–Benign-trigger	**4,000**	**0.0**
Safe–Benign-trigger	2,000	0.8

The 0.8 target for safe vs. benign-with-trigger acknowledges that both are benign but stylistically distinct, encoding a desired similarity rather than a measured geometric outcome.

The encoder is trained with cosine-similarity loss (`losses.CosineSimilarityLoss`, MSE):

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\big(s_i - \cos(\mathbf{e}_1^{(i)}, \mathbf{e}_2^{(i)})\big)^2, \tag{2}$$

where $\mathbf{e}_1, \mathbf{e}_2$ are 768-dim outputs of the fine-tuned encoder and s_i is the target from Table 1.

Recorded training configuration. The model card records batch size 4, learning rate 5×10^{-5}, 100 warm-up steps, and a four-epoch configuration. The encoder script does not explicitly pass the learning rate, so reproducing the model-card configuration requires setting it rather than relying on a library default. The last model-card log entry reports Spearman correlation 0.9040 at step 12,500 (epoch 3.5714); this is not proof of a plateau-triggered early stop. The validation-pool overlap described above limits the interpretation of that correlation.

5.3 XGBoost classifier on the fine-tuned embeddings

The 768-dim embeddings feed an XGBoost classifier (`n_estimators=300`, `max_depth=6`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`, `tree_method=hist`). The classifier was trained on the corpus recorded in `models/injection_aware_mpnet_metadata.json`:

- Safe benign: 2,667 samples;
- Benign-with-trigger: 570 samples;
- Malicious: 413 rows of templated injections and a deepset slice.

The metadata counts rows, not necessarily unique prompts: the safe-text and malicious-text generators repeat templates. The classifier entry point calls `EmbeddingClassifier.train`, which fits all supplied embeddings without a validation split. Its configured parameters do not include `scale_pos_weight`. Thus neither a held-out classifier validation set nor class-ratio loss weighting is established by this training path. The separate `train_on_dataset` method offers a 90/10 split; its existence does not establish that the recorded classifier used that path.

6 Evaluation Evidence and Protocol

6.1 Historical evidence

The count-bearing record used below is `results/injection_aware_mpnet_evaluation.json`, dated 2025-12-16 03:24:39. It identifies Injection-Aware MPNet and threshold 0.764, but lacks model hashes, dataset revisions, and per-example predictions. Its sampling metadata names SaTML, deepset, LLMail, and the local synthetic trigger set. Only the deepset class slices, the synthetic set, and a pooled record have stored confusion counts. We do not infer missing per-dataset counts from the pooled record.

6.2 Metrics and uncertainty

We recompute accuracy, recall, and false-positive rate (FPR) from counts:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}. \tag{3}$$

A metric with zero denominator is N/A, not zero. Table 2 includes nominal 95% Wilson intervals for these proportions. The intervals summarize finite-count uncertainty under an independent Bernoulli assumption; they do not correct training overlap, duplicate prompts, or selection bias. No paired model comparison or McNemar significance claim is made.

7 Results Audit

7.1 Count-backed historical observations

These are pre-remediation observations, not held-out performance. The table is generated directly from the stored confusion counts; it replaces the manually combined per-dataset table. The pooled record covers more samples than the three displayed slices and is shown separately.

Table 2: Historical pre-remediation counts, $\theta = 0.764$. Nominal 95% Wilson intervals in brackets; not held-out performance.

Record	n	**Accuracy**	**Recall**	**FPR**
deepset (benign)	343	100.0% [98.9, 100.0]	N/A	0.0% [0.0, 1.1]
deepset (injections)	203	100.0% [98.1, 100.0]	100.0% [98.1, 100.0]	N/A
Local synthetic triggers	247	100.0% [98.5, 100.0]	N/A	0.0% [0.0, 1.5]
Stored pooled record	4793	97.5% [97.0, 97.9]	97.1% [96.5, 97.6]	0.0% [0.0, 0.6]

Source: `results/injection_aware_mpnet_evaluation.json`, 2025-12-16 03:24:39. The pooled record includes SaTML and LLMail samples whose individual confusion counts are not saved in this file; it is not the sum of the three rows above. The synthetic trigger set is not official NotInject. Intervals do not account for leakage or duplicates.

The local trigger record contains zero false positives in 247 rows, which is not evidence of a zero population error rate. It is also not a result on official NotInject. The separate `results/latest_benchmark_results.txt` summary reports 99.0% overall accuracy for a different dataset mix; it does not establish the former 97.3% headline. Combining figures across these records would obscure their differing denominators and provenance.

7.2 Comparisons, examples, and embedding geometry

We withdraw the prior state-of-the-art and lower-latency claims. The baseline table mixes model generations and published results with no matched input manifest. Truncated qualitative examples and their confidence values lack a linked prediction record and are excluded from empirical results. The claimed IDR change from 2.31 to 0.67 also lacks a model-matched measurement artifact and is withdrawn. A ratio of class-centroid distances alone would not establish that individual examples are correctly clustered.

Likewise, the stored ROC/PR assets do not provide per-example scores and split provenance sufficient to establish independent generalization. They are excluded from this draft. A comparison between doubled and uniform pair sampling, using the same unique texts and held-out predictions, is needed to test whether pair weighting improves over-defense performance.

7.3 Latency evidence

The earlier CDF and boxplot are excluded because `paper/generate_latency_charts.py` generates lognormal random samples rather than loading measured latencies. The benchmark runner times whole prediction batches and assigns batch duration divided by batch size to each item. Quantiles of these amortized times do not measure single-request tail latency. In the embedding-classifier path, this timer includes `predict_proba`, which itself generates embeddings; source inspection does not support a blanket claim that it times only XGBoost.

A replacement measurement must identify hardware, model hashes, input lengths, warm-up, batch size, synchronization, and whether preprocessing is included. Neither the old GPU headline nor a new CPU latency claim is established here.

7.4 BrowseSafe: text extraction, not cross-modality validation

BrowseSafe [Zhang et al., 2025] provides HTML-embedded prompt injections. Our loader removes script and style elements and extracts text, with a raw-prefix fallback for short extracted content. It does not evaluate a rendered browser, CSS visibility, screenshots, or downstream

agent actions. The historical training pool also includes BrowseSafe text. Consequently, the old result cannot establish unseen-domain or cross-modality transfer. The text summary reports 98.0% accuracy and 94.1% recall, while the old manuscript table reported 98.2% and 96.7%; neither is promoted to a new performance claim without matched sample counts and prediction provenance.

8 Bounded Colab Re-evaluation

A new experiment completed on September 11, 2026, using a Tesla T4 in Google Colab. It starts from the public base MPNet encoder rather than the historical checkpoint. Pinned source revisions and capped source prefixes produce 6,370 nonempty rows; one empty LLMail body is recorded as rejected. After removing 1,024 duplicate texts or truncated model inputs and three conflicting-label rows, the manifest contains 2,286 training, 742 validation, and 2,315 test inputs. Normalized text and 384-token input hashes, known LLMail team groups, and synthetic template families do not cross splits. Both training stages use the same training membership. All 339 official NotInject and 247 historical local synthetic prompts are test-only. Unknown source provenance and semantic near-duplicates remain uncontrolled.

The comparison uses base MPNet plus XGBoost, a uniformly sampled pair mixture, and the doubled injection–trigger mixture. Both fine-tuned variants use 14,000 pairs per epoch, four epochs, batch size four, learning rate 5×10^{-5}, 100 warmup steps, and seed 42. The classifier threshold maximizes validation balanced accuracy, with the highest threshold breaking ties, and is saved before test inference. The resulting thresholds are 0.880826, 0.772123, and 0.999148, respectively. Test labels do not select them.

Table 3: New single-seed results on the common 2,315-input test set. Values are percentages at each variant's validation-selected operating point; these are not comparisons at matched recall or false-positive rate.

Variant	Accuracy	Recall	FPR	NotInject FPR
Base MPNet	69.03	45.55	11.28	29.50
Uniform pairs	62.55	48.30	25.50	57.82
Doubled pairs	63.67	64.20	36.78	83.78

The test set contains 1,056 attacks and 1,259 benign inputs. Doubling the pair type increases recall by 15.91 percentage points over uniform pairing, but raises false positives by 11.28 points. On official NotInject it rejects 284/339 benign prompts, versus 196/339 for uniform and 100/339 for base. Thus the sampled experiment does not support the claimed general reduction in over-defense. Base MPNet has the highest pooled accuracy and lowest FPR. One seed and capped data do not establish a population-level ranking or state-of-the-art comparison. Nominal Wilson intervals are retained with the count-bearing artifacts; correlated inputs limit their interpretation.

Fresh synchronized single-input timings include tokenization, GPU encoding, and CPU XGBoost inference. On 150 common inputs after 10 warmups, P50/P95 were 17.31/42.05 ms for base, 20.90/49.44 ms for uniform, and 17.24/45.18 ms for doubled pairs. They exclude HTML preprocessing, request queues, network latency, and downstream application work; they are not production latency.

The downloaded evidence, trained encoders, manifests, predictions, and verification summary are retained in `results/colab_audit_20260911`. All 53 recorded artifact checksums matched; saved classifiers reproduced saved scores from test embeddings. An independent checker also verified split isolation, token hashes, actual pair membership, validation threshold selection, confusion counts, and latency percentiles. This checks artifact consistency and classifier replay;

it does not independently repeat GPU encoder inference or training. Per-source scores can mix official test rows with generated source holdouts; the manifest retains source-split identifiers. The existing TensorTrust cache contributes 121 unique test prompts with unknown upstream sampling provenance. No agent-level attack-success experiment is implied.

8.1 Validation diagnosis and a controlled coverage correction

Inspection of the frozen validation split identified input loss before classification. Of 410 BrowseSafe validation pages, 363 exceed the 382-content-token budget (384 tokens including special tokens). A fixed lexical locator finds instruction phrases only beyond the first window in 52 attack-labelled pages; all three variants miss 39 of these. Manual inspection confirms explicit instructions outside the window in selected cases. In another 21 attack-labelled pages, a phrase matches raw HTML but not extracted text; inspected examples lose payloads in hidden input values, comments, or data attributes. These counts are locator-assisted, not exhaustive attack-span annotations. Quoted instructional discussion also occurs in non-injection pages.

Label and sampling limitations remain. All 136 training trigger examples come from three synthetic template families, while the 68 validation trigger examples share one education family; some substitutions are ungrammatical. No LLMail group reaches validation. A source-only score using training attack prevalence attains pooled validation ROC-AUC 0.779, showing that source recognition can inflate pooled evidence. This does not establish what features any encoder actually learned. A character TF-IDF diagnostic flags 97 validation inputs at cosine similarity at least 0.90 to training text, including seven nearest pairs with opposite labels; similar pages with different inserted payloads need not be mislabeled. Non-injection labels also do not imply general harmlessness, and some isolated competition messages lack the context needed to judge their label. No labels were automatically changed.

A one-factor follow-up retains frozen base MPNet, original training heads, labels, HTML extraction, and classifier parameters. The correction scores overlapping 382-token windows with 96-token overlap and takes the maximum document score. Seeds 17, 42, and 73 vary XGBoost subsampling only; this is not a multi-seed encoder-training experiment. Each method's threshold is selected on validation balanced accuracy and frozen before final inference. The fixed, previously unused BrowseSafe test interval 600–1599 yields 919 inputs (445 attacks, 474 non-injections) after excluding 81 exact or highly similar inputs against prior raw data and within the fresh holdout. The similarity cutoff is 0.90; unknown website and semantic dependence remain. No seed or operating point is selected using the final scores.

Table 4: Fresh BrowseSafe follow-up. FPR values are percentages at validation-selected thresholds. Both methods share the same classifier within each seed.

Seed	Base AUC	Windowed AUC	Base FPR	Windowed FPR
17	0.5398	0.5471	12.24	45.99
42	0.5330	0.5430	6.96	36.08
73	0.5331	0.5412	22.78	60.34

The predefined direction-consistency criterion holds, but the AUC gains are only 0.0073–0.0100 and all nominal paired bootstrap 95% intervals include zero. The 1,000-resample, label-stratified bootstrap assumes independent documents and cannot account for unknown website dependence. Windowing raises recall from 11.24–30.11% to 43.15–65.39% across seeds while substantially increasing false positives. It does not provide a useful detector on this slice and cannot recover payloads removed by HTML extraction. All 30 recorded artifact hashes matched; window token inputs, saved classifier predictions, and validation thresholds were verified. Seed 42 exactly reproduced the original base validation scores. GPU encoder inference was not

independently repeated. Detailed evidence is retained in `paper/VALIDATION_DIAGNOSIS.md` and `results/colab_window_followup_20260911/window_followup`.

8.2 Supervised encoder pilot across training seeds

A subsequent controlled experiment compares a frozen MPNet encoder with full encoder fine-tuning through the same linear binary-classification head. Both arms preserve unchanged raw HTML, including comments, attributes, and scripts, and score all overlapping 382-content-token windows. The maximum window logit gives one page-level binary cross-entropy loss; page labels are not assigned to individual windows. The fine-tuned arm selects the maximum without gradients and recomputes that window with gradients. Dropout is disabled in both arms, making this a max-loss subgradient apart from finite-precision batching differences.

The pilot uses 256 training pages (128 per class), 192 validation pages (96 per class), and 447 previously unscored holdout pages (223 attacks, 224 non-injections), all from the same pinned BrowseSafe revision. Training and validation retain their prior partition membership. A fixed hash order samples eligible pages after character TF-IDF overlap rejection at cosine similarity 0.90. The fresh official-test interval 1600–2099 excludes 53 candidates against prior data or other new holdout pages. There are 18,409 training, 14,823 validation, and 34,189 holdout windows. A bounded label review quarantines one ambiguous training page without relabeling it; other source labels remain machine-generated supervision. Single-source sampling removes across-source label imbalance from this comparison but cannot remove unknown website or template dependence.

Seeds 17, 42, and 73 vary head initialization and training order, with matched initialization and order between arms. Unlike the earlier window follow-up, these include three actual encoder-training runs. Each arm receives at most two epochs, four pages per optimizer step, AdamW weight decay 0.01, and gradient clipping at 1. Encoder and head learning rates are 10^{-5} and 10^{-3}. Execution uses an NVIDIA A100-SXM4-40GB with float16 autocast and float32 parameters. Validation maximizes recall at empirical FPR at most 5%, using ROC-AUC and the earliest epoch to break checkpoint ties. This budget permits at most four validation false positives; it is not a guarantee on new data. All six checkpoints and thresholds are frozen before any holdout scoring.

Table 5: Supervised encoder pilot on 447 fresh BrowseSafe pages. Recall and FPR are percentages at each model's validation-frozen threshold. F denotes frozen encoder; T denotes fine-tuned encoder.

Seed	F AUC	T AUC	F recall	T recall	F FPR	T FPR
17	0.5316	0.8353	7.17	61.88	6.25	2.23
42	0.5264	0.4668	5.38	10.31	4.02	8.48
73	0.5158	0.8584	5.83	51.12	5.36	0.45

Seeds 17 and 73 improve recall while reducing false positives. Seed 42 has worse AUC and exceeds the exploratory FPR budget on the holdout. The experiment therefore does not establish a reliable improvement across training seeds. Nominal paired 95% bootstrap intervals for the AUC differences are $[0.2483, 0.3558]$, $[-0.1174, -0.0021]$, and $[0.2814, 0.3963]$ for seeds 17, 42, and 73. Figure 1 also shows the primary recall differences alongside FPR differences. The 1,000 class-stratified document resamples assume independent pages and omit unknown website/template dependence and label uncertainty. The unchanged MPNet–XGBoost baseline from the earlier audited run obtains AUC 0.5695, recall 12.56%, and FPR 5.80% on these same pages, but its different training data and input extraction make it a contextual reference, not the controlled encoder ablation. The short training budget does not establish convergence or the best achievable performance of either arm. This holdout is now consumed; subsequent model

choices require validation and another untouched final evaluation. No standalone security-gate claim is supported. Protocol and evidence are documented in `paper/SUPERVISED_PILOT.md`. All 81 recorded hashes, full token coverage, saved linear-head and historical classifier predictions, validation selections, and paired training orders passed local verification. GPU encoder training and inference were not independently repeated.

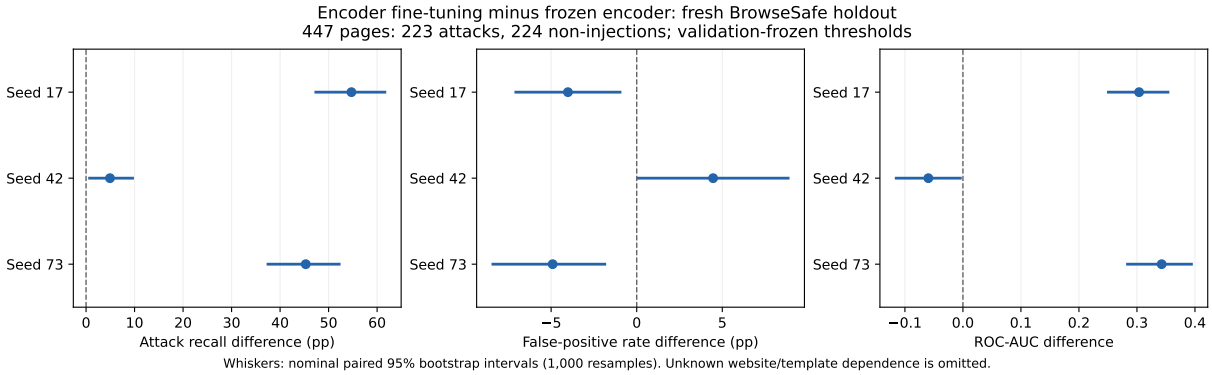


Figure 1: Fine-tuned minus frozen encoder on the fresh holdout. Positive recall and AUC differences and negative FPR differences are favorable. Whiskers are nominal paired 95% document-bootstrap intervals; unknown website/template dependence is not represented. Rates use each model's validation-frozen threshold.

8.3 Six-epoch supervised schedule follow-up

A training/validation-only diagnosis found that seed 42's mean training loss changed from 0.6946 to 0.6972 over the first two epochs, despite nonzero encoder updates. Its validation AUC improved in epoch two, but recall under the FPR budget fell, retaining epoch one. The selected checkpoint's median top-two-window logit gap was 0.00075, versus 0.0421 and 0.0454 for the other seeds; these checkpoints came from different epochs. These observations motivate a longer schedule without establishing a label, pooling or optimizer defect.

The follow-up completes six fixed epochs, retaining the best validation checkpoint, instead of at most two epochs with early stopping. It keeps the same 256 training and 192 validation pages, architecture, objective, raw-HTML window coverage, learning rates, optimizer, precision, seeds, initialization and page orders. Frozen-encoder controls remain. A separate 453-page holdout contains 217 attacks and 236 non-injections (34,449 windows), from the pinned BrowseSafe official-test interval 2100–2599. Before scoring, 47 of 500 candidates were excluded by normalized raw/extracted/first-window matches and character-TF-IDF cosine similarity at least 0.90 against 7,870 prior inputs or earlier retained candidates. This lexical screen does not establish website or template independence. Validation retains the same recall/FPR, AUC and earliest-epoch selection rule. All twelve prior/new models and thresholds are frozen before evaluation; no test score is used to select a seed.

Numerical amendment. The first attempt stopped at seed 17's update 181: nonfinite-gradient clipping raised before GradScaler could reduce its scale. No holdout evaluation had begun. A protocol amendment, frozen before test scoring, clears invalid gradients, lets GradScaler skip the invalid attempt and back off, then retries backward on the same retained forward graph. Persistent nonfinite gradients abort after at most eight retries. All six trials restart from initialization and complete 384 finite optimizer updates without omitting a batch. Fine-tuned seeds 17/42/73 retry two batches each, with 2/4/2 invalid attempts; frozen arms have none. Forced-overflow recovery and persistent-nonfinite rejection checks pass. All first-two-epoch validation scores, decisions and page orders match the original runs exactly. This is a schedule

comparison with a disclosed numerical repair, not an unamended one-factor experiment. The overflow occurs beyond the original two-epoch budget and does not explain the earlier seed-42 result.

Table 6: Prior versus revised fine-tuned schedules on the same 453-page BrowseSafe holdout. P denotes the prior schedule; R denotes six epochs with the numerical amendment. Recall and FPR are percentages at validation-frozen thresholds.

Seed	P AUC	R AUC	P recall	R recall	P FPR	R FPR
17	0.8128	0.8341	59.45	69.12	0.42	2.54
42	0.4532	0.8732	4.61	66.36	9.32	4.66
73	0.8360	0.8660	45.16	67.74	0.42	2.97

All revised fine-tuned selections are epoch six. Seed 42's loss falls to 0.4219 and its validation recall reaches 76.04%, versus 7.29% at the prior selection. This supports insufficient training duration as a contributor in this setup, without proving convergence or a universal cause. On the holdout, the declared directional criterion passes: each seed improves recall and has empirical FPR at most 5%. Revised true-positive/false-positive counts are 150/6, 144/11 and 147/7, versus prior counts 129/1, 10/22 and 98/1. Nominal paired 95% intervals for recall gains are $[5.99, 13.82]$, $[53.92, 68.66]$ and $[16.59, 29.03]$ percentage points (1,000 class-stratified document bootstrap resamples). False positives increase for seeds 17 and 73; the seed-17 AUC-difference interval crosses zero and the seed-42 FPR-difference interval reaches zero (Figure 2). Revised FPR Wilson 95% upper bounds are 5.43%, 8.15% and 5.99%, so population FPR below 5% is unestablished. These nominal intervals omit website/template dependence, label uncertainty, multiplicity and variation across further training seeds.

The six-epoch frozen controls have AUC 0.5066/0.5435/0.5066, recall 5.07/8.29/3.23% and FPR 7.20/5.08/3.81% for seeds 17/42/73. The preserved historical base MPNet–XGBoost reference obtains AUC 0.5699, recall 14.29% and FPR 3.39%; its different training data and extraction make it contextual, not the paired schedule comparator.

Interrupted execution and verification. The first amended A100 run finished training and package replay but lost its Colab assignment during evaluation; keep-alive logs recorded OAuth scope-refresh failures. After repairing refresh under the same grant, a new A100 repeated the unchanged frozen protocol for artifact recovery. All six original checkpoint files reproduced byte for byte, as did both retrieved revised checkpoints (17/42); all 36 captured validation summaries matched. The interrupted seed-73 checkpoint was lost before retrieval, precluding a byte comparison to that file. Its recovered checkpoint is saved and verified. The holdout was untouched before initial choices but partially consumed by the interrupted evaluation. The final evaluation is a fixed remeasurement, not a new model-selection experiment; no scientific choices changed after test output.

All 238 recorded artifact hashes, full token coverage, saved-head and historical classifier predictions, selection rules, freezes, histories, optimizer logs and recovery evidence pass local verification. The six prior/new fine-tuned inference packages replay all 192 validation pages on A100 with identical decisions and maximum score error 5.96×10^{-8}. Local checks do not independently repeat full GPU encoder computation; prior fine-tuned head replay uses saved winning embeddings. Three revised portable inference packages, complete notebooks and evidence are retained in `results/supervised_schedule_20260911`; the protocol and runnable verification command are in `paper/SUPERVISED_SCHEDULE.md`. Small, weakly labelled, single-source data and reused validation remain limitations. This consumed holdout must not inform further model choices. The result does not establish downstream security effectiveness or validate the original contrastive over-defense claim.

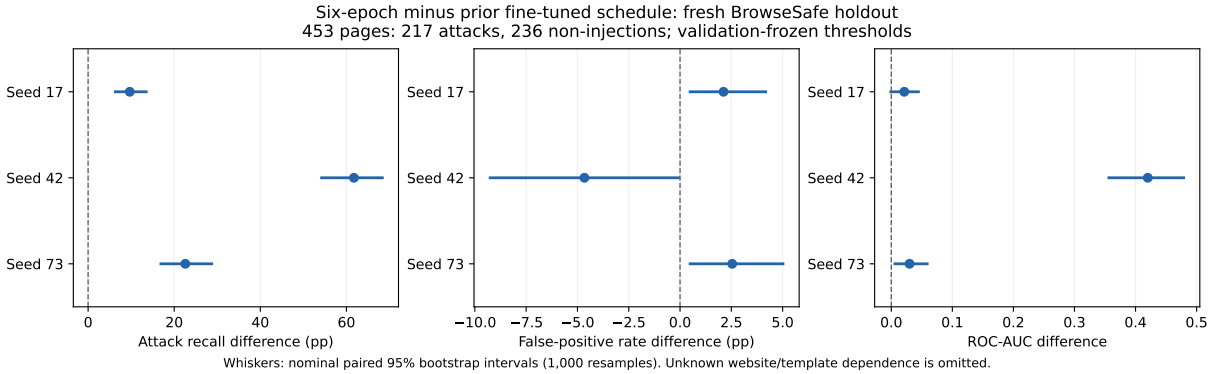


Figure 2: Six-epoch minus prior fine-tuned schedule on the separate 453-page holdout. Positive recall and AUC differences and negative FPR differences are favorable. Whiskers are nominal paired 95% bootstrap intervals with the dependence and label limitations stated in the text.

9 Discussion and Threats to Validity

Leakage and remediation status. The original encoder loads evaluation-source text before pair construction, and its validation tail remains in the training pool. The classifier's separate training entry point also consumes local synthetic trigger text and deepset injections that are used for evaluation. A split applied only to classifier embeddings cannot undo prior encoder exposure.

The later source change at commit `3d1bba8`, merged in `1be03c1`, subtracts a benchmark holdout from the encoder pools and splits internal validation before pair construction. This is a code change, not evidence of a retrained checkpoint. It also leaves the separate classifier training entry point unchanged, omits the local synthetic benchmark from the holdout, and splits row positions without enforcing unique normalized texts. Its verification script checks a reconstructed set after subtraction rather than the actual texts used by a trained model. A model-bound, independent verification remains necessary.

Remaining re-evaluation requirements. Section 8 provides a new checkpoint-bound experiment with shared, deduplicated splits and saved predictions for both training stages. The historical checkpoint and results remain contaminated; the new run does not retroactively validate them. Broader evidence still requires replication of the contrastive-pair comparison across encoder-training seeds, stronger provenance and semantic near-duplicate controls, complete benchmark coverage, and evaluation of operational or downstream security outcomes. The original local weight-file deletion has been preserved; new encoders are stored separately with the downloaded audit evidence.

Scope of security conclusions. Single-text classification does not measure downstream attack success, agent task utility, authentication, quarantine effectiveness, or resistance to multi-turn and adaptive attacks. The 36-example GCG/AutoDAN-labelled record identifies an older BIT classifier and counts detector bypasses; it is not a matched adaptive attack evaluation for Injection-Aware MPNet. No traceable TensorTrust result for the claimed 83.2% was located, so that number is withdrawn. AgentDojo artifacts exist, but a complete evaluation bound to this MPNet checkpoint is not established by their presence.

10 Conclusion

Injection-Aware MPNet implements a testable approach to prompt-injection detection: emphasize injection–benign-trigger sentence pairs during encoder training and classify the resulting

embeddings. The available historical evidence does not establish held-out accuracy, a causal reduction in over-defense, state-of-the-art performance, or production latency. The new bounded experiment improves recall with doubled pairs at the cost of substantially more false positives, including on official NotInject. It does not support a general reduction in over-defense. The revised paper also records a controlled window-coverage follow-up: small AUC changes across three classifier seeds accompany substantially more false positives on a fresh BrowseSafe slice. A supervised raw-HTML encoder pilot improves two of three training seeds, but the third has worse AUC and exceeds its FPR budget on a fresh holdout. Extending supervised training to six epochs with a disclosed numerical repair improves recall across all three seeds on a separate 453-page holdout, reaching 66.36–69.12% recall and 2.54–4.66% empirical FPR. Two seeds incur more false positives; population FPR below 5%, broader generalization and the original contrastive over-defense claim remain unestablished. These experiments do not support standalone security-gate use. The revised paper records supported implementation details, historical counts, and the new checkpoint-bound measurements while making their limitations explicit. Repository materials are available at `https://github.com/goodwiins/prompt-injection-defense`; the new audit retains checkpoints, manifests, predictions, and a runnable verification command, while broader security conclusions remain unestablished.

References

Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *Proceedings of the 34th USENIX Security Symposium*, 2025a. URL `https://arxiv.org/abs/2402.06363`. arXiv:2402.06363.

Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. SecAlign: Defending against prompt injection with preference optimization. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025b. URL `https://arxiv.org/abs/2410.05451`. arXiv:2410.05451.

Falong Fan and Xi Li. PeerGuard: Defending multi-agent systems against backdoor attacks through mutual reasoning. *arXiv preprint arXiv:2505.11642*, 2025. URL `https://arxiv.org/abs/2505.11642`.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023. URL `https://arxiv.org/abs/2302.12173`.

Donghyun Lee and Mo Tiwari. Prompt infection: Llm-to-llm prompt injection within multi-agent systems. *arXiv preprint arXiv:2410.07283*, 2024. URL `https://arxiv.org/abs/2410.07283`.

Hao Li and Xiaogeng Liu. InjecGuard: Benchmarking and mitigating over-defense in prompt injection guardrail models. *arXiv preprint arXiv:2410.22770*, 2024. URL `https://arxiv.org/abs/2410.22770`. ACL 2025.

Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022. URL `https://arxiv.org/abs/2211.09527`.

Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. PromptArmor: Simple yet effective prompt injection defenses. *arXiv preprint arXiv:2507.15219*, 2025. URL `https://arxiv.org/abs/2507.15219`.

Kaiyuan Zhang, Mark Tenenholtz, Kyle Polley, Jerry Ma, Denis Yarats, and Ninghui Li. BrowseSafe: Understanding and preventing prompt injection within AI browser agents. arXiv:2511.20597, 2025. URL https://arxiv.org/abs/2511.20597.